

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 1
“SKEMA PEMROSESAN SEKUENSIAL”



DISUSUN OLEH:
Muhammad Nabil Raissa P
109082500127
S1IF-13-02
DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

DASAR TEORI

Bab enam ini berisi dua modul materi dan soal pengayaan, praktikan bisa melakukan eksplorasi secara mandiri di luar jam praktikum. Praktikan bisa bertanya kepada dosen terkait materi dan soal pada modul ini apabila terdapat hal-hal yang kurang dimengerti. Adapun modul yang akan di bahas adalah

a) Modul 17. Skema Pemrosesan Sekuensial Pada modul ini diperkenalkan skema pemrosesan sekuensial yang bisa digunakan untuk memperkuat pemahaman struktur kontrol perulangan. Penggunaan struktur kontrol memungkinkan praktikan untuk bisa menyelesaikan kasus-kasus perulangan berdasarkan pola-pola yang diperkenalkan pada skema pemrosesan sekuensial.

b) Modul 18. Mesin Abstract Modul 18 ini memperkenalkan penggunaan konsep mesin abstract. Suatu model komputasi atau program (mesin) dibuat di berdasarkan model komputasi/program/mesin yang telah ada.

SOAL LATIHAN

Statement perulangan

1.

Source Code:

```
package main

import (
    "fmt"
)

func main() {
    var num, sum float64
    var count int

    for {
        fmt.Scan(&num)
        if num == 9999 {
            break
        }
        sum += num
        count++
    }

    if count > 0 {
        rerata := sum / float64(count)
        fmt.Printf("Rerata: %.2f\n", rerata)
    } else {
        fmt.Println("Belum ada data yang dimasukkan.")
    }
}
```

Output:

Deskripsi Program:

2.

Source Code:

```
package main
```

```

import (
    "fmt"
)

func main() {
    var x string
    var n int

    fmt.Scan(&x)
    fmt.Scan(&n)

    data := make([]string, n)
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    var found bool
    var positions []int
    var count int

    for i := 0; i < n; i++ {
        if data[i] == x {
            found = true
            positions = append(positions, i+1)
            count++
        }
    }

    if found {
        fmt.Println("a. Apakah string x ada? Ya.")
    } else {
        fmt.Println("a. Apakah string x ada? Tidak.")
    }

    if found {
        fmt.Print("b. Ditemukan pada posisi ke: ")
        for i, pos := range positions {
            if i > 0 {
                fmt.Print(", ")
            }
            fmt.Print(pos)
        }
        fmt.Println()
    } else {
        fmt.Println("b. Ditemukan pada posisi ke: -")
    }

    fmt.Printf("c. Ada berapakah string x? %d buah.\n", count)

    if count >= 2 {

```

```
        fmt.Println("d. Adakah sedikitnya dua string x? Ya.")
    } else {
        fmt.Println("d. Adakah sedikitnya dua string x? Tidak.")
    }
}
```

Output:

Deskripsi Program:

3.

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    var n int
    fmt.Scan(&n)

    var tetesanA, tetesanB, tetesanC, tetesanD int

    for i := 0; i < n; i++ {
        x := rand.Float64()
        y := rand.Float64()

        if x < 0.5 && y < 0.5 {
            tetesanA++
        } else if x >= 0.5 && y < 0.5 {
            tetesanB++
        } else if x >= 0.5 && y >= 0.5 {
            tetesanC++
        } else if x < 0.5 && y >= 0.5 {
            tetesanD++
        }
    }

    curahA := float64(tetesanA) * 0.0001
    curahB := float64(tetesanB) * 0.0001
    curahC := float64(tetesanC) * 0.0001
    curahD := float64(tetesanD) * 0.0001
}
```

```

fmt.Printf("Curah hujan daerah A: %.4f milimeter\n", curahA)
fmt.Printf("Curah hujan daerah B: %.4f milimeter\n", curahB)
fmt.Printf("Curah hujan daerah C: %.4f milimeter\n", curahC)
fmt.Printf("Curah hujan daerah D: %.4f milimeter\n", curahD)
}

```

Output:

```

func main() {
    var n int
    fmt.Scan(&n)

    var tetesanA, tetesanB, tetesanC, tetesanD int

    for i := 0; i < n; i++ {
        x := rand.Float64()
        y := rand.Float64()

        if x < 0.5 && y < 0.5 {
            tetesanA++
        } else if x >= 0.5 && y < 0.5 {
            tetesanB++
        } else if x >= 0.5 && y >= 0.5 {
            tetesanC++
        } else if x < 0.5 && y >= 0.5 {
            tetesanD++
        }
    }
}

```

```

PS C:\Users\nabil\OneDrive\Dokumen\semester2alpro> go run "c:\Users\nabil\OneDrive\Dokumen\semester2alpro\2A\
ggg 17-18\3.go"
10000000
Curah hujan daerah A: 249.7800 milimeter
Curah hujan daerah B: 250.2167 milimeter
Curah hujan daerah C: 249.9268 milimeter
Curah hujan daerah D: 250.0765 milimeter
PS C:\Users\nabil\OneDrive\Dokumen\semester2alpro>

```

Deskripsi Program:

aku bikin simulasi hujan dengan nge-looping sebanyak n tetesan air yang di-input-in. Di dalem looping-nya, aku pake fungsi rand.Float64() buat nge-generate titik koordinat x sama y secara acak buat tiap tetesnya. Terus, aku tinggal bikin kondisi *if-else* buat ngecek airnya jatuh di daerah mana (A, B, C, atau D) pake patokan batas tengah 0.5. Begitu looping-nya kelar dan semua tetesan udah kehitung, total tetesan di masing-masing area itu tinggal aku kaliin 0.0001 biar dapet angka curah hujannya dalam bentuk milimeter, trus langsung aku *print* deh ke terminal pake format 4 angka di belakang koma

4.

Source Code:

```

package main

import (

```

```

    "fmt"
    "math"
)

func main() {
    var n int
    fmt.Print("N suku pertama: ")
    fmt.Scan(&n)

    piPrev := 0.0
    piCurr := 0.0

    for i := 1; i <= n; i++ {
        if i > 1 {
            diff := math.Abs(piCurr - piPrev)
            if diff <= 0.00001 {
                fmt.Printf("Hasil PI: %.10f\n", piPrev)
                fmt.Printf("Hasil PI: %.10f\n", piCurr)
                fmt.Printf("Pada i ke: %d\n", i)
                return
            }
        }

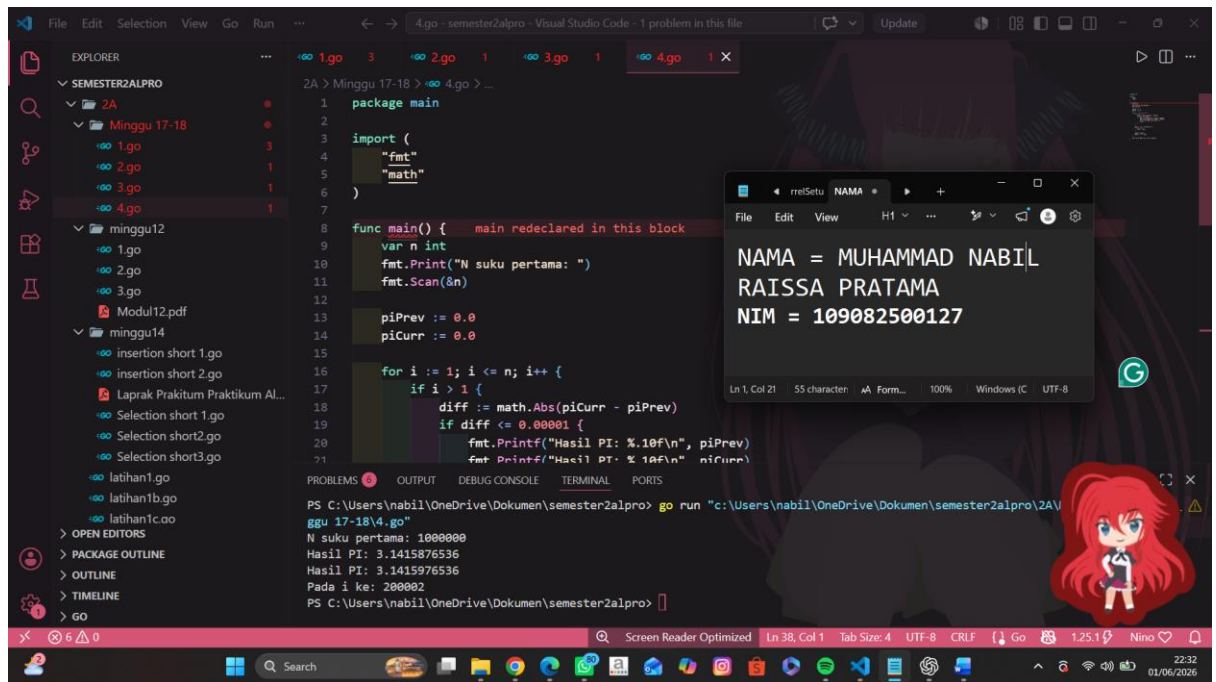
        term := 1.0 / float64(2*i-1)
        if i%2 == 0 {
            term = -term
        }

        piPrev = piCurr
        piCurr += 4 * term
    }

    fmt.Printf("Hasil PI: %.7f\n", piCurr)
}

```

Output:



Deskripsi Program:

kodingan ini tuh dibikin buat ngitung nilai PI pake rumus deret Leibniz, tapi versinya udah di-*upgrade* biar lebih pinter. Sistem kerjanya, program bakal terus-terusan nge-*looping* buat ngitung suku demi suku sambil nyimpen nilai PI yang sekarang sama nilai PI selangkah sebelumnya. Nah, pas lagi asik muter, dia bakal selalu ngecek selisih antara dua nilai tersebut. Kalo ternyata selisihnya udah super tipis alias kurang dari atau sama dengan 0.00001, programnya auto ngerem mendadak trus langsung nyetak dua nilai PI terakhir plus info di putaran ke berapa dia berhenti, jadi programnya nggak perlu capek-capek nungguin *looping*-nya beres sampai jutaan kali

5.

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    var totalTopping int
    fmt.Print("Banyak Topping: ")
    fmt.Scan(&totalTopping)

    toppingDalamPizza := 0
    xc, yc := 0.5, 0.5
    r := 0.5
```



```

for i := 0; i < totalTopping; i++ {
    x := rand.Float64()
    y := rand.Float64()

    jarakKuadrat := (x-xc)*(x-xc) + (y-yc)*(y-yc)

    if jarakKuadrat <= r*r {
        toppingDalamPizza++
    }
}

fmt.Printf("Topping pada Pizza: %d\n", toppingDalamPizza)

pi := 4.0 * float64(toppingDalamPizza) / float64(totalTopping)
fmt.Printf("PI : %.10f\n", pi)
}

```

Output :

```

1 package main
2
3 import (
4     "fmt"
5     "math/rand"
6 )
7
8 func main() {
9     var totalTopping int
10    fmt.Print("Banyak Topping: ")
11    fmt.Scan(&totalTopping)
12
13    toppingDalamPizza := 0
14    xc, yc := 0.5, 0.5
15    r := 0.5
16
17    for i := 0; i < totalTopping; i++ {
18        x := rand.Float64()
19        y := rand.Float64()
20
21        jarakKuadrat := (x-yc)*(x-yc) + (y-yc)*(y-yc)
22
23        if jarakKuadrat <= r*r {
24            toppingDalamPizza++
25        }
26    }
27
28    pi := 4.0 * float64(toppingDalamPizza) / float64(totalTopping)
29    fmt.Printf("PI : %.10f\n", pi)
30 }

```

Output:

```

Banyak Topping: 1234567
Topping pada Pizza: 969523
PI : 3.1412568131

```

Deskripsi :

ceritanya aku bikin simulasi buat ngitung nilai PI pake lemparan *topping*. Sistemnya, aku terima *input* total *topping* dulu, terus aku bikin *looping* buat nge-generate posisi jatuhnya masing-masing *topping* secara acak pake fungsi `rand.Float64()`. Di dalam *looping* itu, aku pasang rumus persamaan lingkaran buat ngecek apakah koordinat

topping-nya beneran mendarat di atas pizzanya atau malah meleset jatuh ke pojokan kotak wadahnya. Setelah beres ngitung semua *topping* yang sukses nempel di pizza, angkanya tinggal aku kali 4 terus dibagi sama total *topping* tebaran awal buat dapet perkiraan nilai PI-nya. Serunya, karena program ini murni jalan pake sistem *random*, hasil tebakan PI-nya bakal selalu beda tipis tiap kali programnya aku jalanin

DAFTAR PUSTAKA